

activity-03-code

September 15, 2026

1 Activity 03: Fundamentals of Cryptography

1.1 Import External Libraries

```
[1]: import re
import math
import time
import string
import statistics
from collections import Counter

import pandas as pd

pd.set_option("display.max_colwidth", None)
```

1.2 Problem 1: Cryptanalysis

1.2.1 Cipher Text and English Letter Frequencies

```
[2]: CIPHER_TEXT = "PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALQSKR. QDFP FP ZK LIUBROJZK MOLTROE."
ALPHABET = string.ascii_uppercase

# Relative frequency (%) of each letter in typical English text
ENGLISH_FREQ = {
    "E": 12.70, "T": 9.06, "A": 8.17, "O": 7.51, "I": 6.97, "N": 6.75, "S": 6.33,
    "H": 6.09, "R": 5.99, "D": 4.25, "L": 4.03, "C": 2.78, "U": 2.76, "M": 2.41,
    "W": 2.36, "F": 2.23, "G": 2.02, "Y": 1.97, "P": 1.93, "B": 1.29, "V": 0.98,
    "K": 0.77, "J": 0.15, "X": 0.15, "Q": 0.10, "Z": 0.07,
}
```

1.2.2 Download the English Dictionary

Download the ENABLE word list (172,823 English words) from the following link: <https://raw.githubusercontent.com/dolph/dictionary/master/enable1.txt>

```
[3]: with open("enable1.txt", "r") as f:
    ENGLISH_WORDS = {w.strip().upper() for w in f if w.strip()}

print(f"Dictionary size: {len(ENGLISH_WORDS):,} words")
```

Dictionary size: 172,823 words

1.2.3 Task (a): Frequency of Letters

```
[4]: letters = [c for c in CIPHER_TEXT if c in ALPHABET]
letter_counts = Counter(letters)

freq_df = pd.DataFrame(
```

```

    [(c, n, 100 * n / len(letters)) for c, n in letter_counts.most_common()],
    columns=["cipher letter", "count", "frequency (%)"],
)
freq_df.index += 1

print(f"Total letters:    {len(letters)}")
print(f"Distinct letters: {len(letter_counts)}")
print(f"Never used:       {' '.join(sorted(set(ALPHABET) - set(letter_counts)))}")
freq_df.round(2)

```

Total letters: 59
 Distinct letters: 20
 Never used: G H N V W Y

```

[4]:   cipher letter  count  frequency (%)
      1           P      7         11.86
      2           R      6         10.17
      3           O      6         10.17
      4           F      6         10.17
      5           Q      5          8.47
      6           L      4          6.78
      7           S      3          5.08
      8           A      3          5.08
      9           Z      3          5.08
     10           K      3          5.08
     11           C      2          3.39
     12           D      2          3.39
     13           J      2          3.39
     14           X      1          1.69
     15           I      1          1.69
     16           U      1          1.69
     17           B      1          1.69
     18           M      1          1.69
     19           T      1          1.69
     20           E      1          1.69

```

```

[5]: # Top three distinct counts -- several letters can share the same count
top_counts = sorted(set(letter_counts.values()), reverse=True)[:3]
for rank, n in enumerate(top_counts, start=1):
    tied = [c for c, m in letter_counts.most_common() if m == n]
    print(f"Rank {rank}: {' '.join(tied)} -> {n} times ({100 * n / len(letters):.2f}%)")

```

Rank 1: P -> 7 times (11.86%)
 Rank 2: R, O, F -> 6 times (10.17%)
 Rank 3: Q -> 5 times (8.47%)

```

[6]: # Side-by-side ranking: cipher letters vs. English letters
english_rank = sorted(ENGLISH_FREQ, key=ENGLISH_FREQ.get, reverse=True)

compare_df = pd.DataFrame(
    {
        "cipher letter": freq_df["cipher letter"].values[:10],
        "cipher (%)": freq_df["frequency (%)"].values[:10].round(2),
        "English letter": english_rank[:10],
        "English (%)": [ENGLISH_FREQ[c] for c in english_rank[:10]],
    },
    index=range(1, 11),
)
compare_df

```

```
[6]:
```

	cipher letter	cipher (%)	English letter	English (%)
1	P	11.86	E	12.70
2	R	10.17	T	9.06
3	O	10.17	A	8.17
4	F	10.17	O	7.51
5	Q	8.47	I	6.97
6	L	6.78	N	6.75
7	S	5.08	S	6.33
8	A	5.08	H	6.09
9	Z	5.08	R	5.99
10	K	5.08	D	4.25

1.2.4 Task (b): Frequency of Words and Word Patterns

Short words are the weakest point of a substitution cipher: English has only a handful of common two- and three-letter words, and a word's *letter pattern* (which positions repeat a letter) survives a monoalphabetic substitution unchanged.

```
[7]: COMMON_2_LETTER = ["OF", "TO", "IN", "IT", "IS", "BE", "AS", "AT", "SO", "WE", "HE", "BY",
                        "OR", "ON", "DO", "IF", "ME", "MY", "UP", "AN", "GO", "NO", "US", "AM"]
COMMON_3_LETTER = ["THE", "AND", "FOR", "ARE", "BUT", "NOT", "YOU", "ALL", "ANY", "CAN",
                    "HAD", "HER", "WAS", "ONE", "OUR", "OUT", "HAS", "HIS", "HOW", "WHO"]

def word_pattern(word):
    """Letter-repetition pattern of a word, e.g. 'PROVERB' -> 'A.B.C.D.E.B.F'."""
    first_seen = {}
    return "".join(first_seen.setdefault(c, ALPHABET[len(first_seen)]) for c in word)

cipher_words = re.findall(r"[A-Z]+", CIPHER_TEXT)
word_counts = Counter(cipher_words)

word_df = pd.DataFrame(
    [(w, len(w), n, word_pattern(w)) for w, n in word_counts.items()],
    columns=["cipher word", "length", "count", "pattern"],
).sort_values(["length", "count"], ascending=[True, False], ignore_index=True)
word_df.index += 1
word_df
```

```
[7]:
```

	cipher word	length	count	pattern
1	FP	2	2	A.B
2	LA	2	1	A.B
3	ZK	2	1	A.B
4	QDR	3	1	A.B.C
5	QDFP	4	1	A.B.C.D
6	AFOPQ	5	1	A.B.C.D.E
7	CZSPR	5	1	A.B.C.D.E
8	MOLTROE	7	1	A.B.C.D.E.B.F
9	PRCSOFQX	8	1	A.B.C.D.E.F.G.H
10	LIUBROJZK	9	1	A.B.C.D.E.F.G.H.I
11	JFPALQSKR	10	1	A.B.C.D.E.F.G.H.I.J

```
[8]: print("Repeated words:      ", {w: n for w, n in word_counts.items() if n > 1})
print("Two-letter words:      ", [w for w in word_counts if len(w) == 2])
print("Three-letter words:    ", [w for w in word_counts if len(w) == 3])
print("Words sharing 'QD':    ", [w for w in word_counts if w.startswith("QD")])
print("Words with a repeated letter:", [w for w in word_counts if len(set(w)) < len(w)])
```

Repeated words: {'FP': 2}
 Two-letter words: ['FP', 'LA', 'ZK']
 Three-letter words: ['QDR']
 Words sharing 'QD': ['QDR', 'QDFP']
 Words with a repeated letter: ['MOLTROE']

1.2.5 Task (c): Cracking the Cipher Text

Each step guesses one cipher word. `candidates()` lists every dictionary word that has the same letter pattern **and** agrees with the letters already solved, so each guess can be checked against the dictionary before it is accepted.

```
[9]: def partial_decrypt(ciphertext, key):
    """Replace solved cipher letters with their plaintext letter; unsolved letters become '_'."""
    return "".join(key.get(c, "_") if c in ALPHABET else c for c in ciphertext)

def candidates(cipher_word, key):
    """Dictionary words matching the cipher word's pattern and consistent with the partial key."""
    pattern = word_pattern(cipher_word)
    used = set(key.values())
    return sorted(
        w for w in ENGLISH_WORDS
        if len(w) == len(cipher_word)
        and word_pattern(w) == pattern
        and all(key[c] == p if c in key else p not in used for c, p in zip(cipher_word, w))
    )
```

```
[10]: # (cipher word, guessed plaintext word) in the order the guesses were made
GUESSES = [
    ("QDR", "THE"),          # only 3-letter word -> most common English word
    ("FP", "IS"),           # repeated 2-letter word, also ends QDFP
    ("QDFP", "THIS"),       # TH__ with distinct letters
    ("PRCSOFQX", "SECURITY"),
    ("AFOPQ", "FIRST"),
    ("CZSPR", "CAUSE"),
    ("LA", "OF"),
    ("JFPALQSKR", "MISFORTUNE"),
    ("ZK", "AN"),
    ("MOLTROE", "PROVERB"),
    ("LIUBROJZK", "OLDGERMAN"), # two words written without a space
]

key = {}
log = []
start = time.perf_counter()
for step, (cipher_word, guess) in enumerate(GUESSES, start=1):
    cands = candidates(cipher_word, key)
    log.append((step, cipher_word, len(cands), ", ".join(cands[:6]) + ("..." if len(cands) > 6
    else ""), guess))
    key.update(zip(cipher_word, guess))
    print(f"Step {step:>2} {cipher_word:>10} -> {guess:<10} | {partial_decrypt(CIPHER_TEXT, _
    key)}")
elapsed = time.perf_counter() - start

print(f"\nDictionary-assisted cracking time: {elapsed:.3f} seconds")
pd.DataFrame(log, columns=["step", "cipher word", "#candidates", "candidates", "chosen"]).
    set_index("step")
```

```
Step 1      QDR -> THE      | _E____T_ __ THE ____T ____E __ ____T__E.
TH__ __ __ ____E____ ____E__.
```

```

Step 2      FP -> IS      | SE__IT_ IS THE _I_ST __SE __ _IS__T__E.
THIS IS __ ____E____ _E___.

Step 3      QDFP -> THIS  | SE__IT_ IS THE _I_ST __SE __ _IS__T__E.
THIS IS __ ____E____ _E___.

Step 4      PRCSOFQX -> SECURITY | SECURITY IS THE _IRST C_USE __ _IS__RTU_E.
THIS IS __ ____ER____ _R__ER_.

Step 5      AFOPQ -> FIRST  | SECURITY IS THE FIRST C_USE _F _ISF_RTU_E.
THIS IS __ ____ER____ _R__ER_.

Step 6      CZSPR -> CAUSE   | SECURITY IS THE FIRST CAUSE _F _ISF_RTU_E.
THIS IS A_ ____ER_A_ _R__ER_.

Step 7      LA -> OF        | SECURITY IS THE FIRST CAUSE OF _ISFORTU_E.
THIS IS A_ O____ER_A_ _RO_ER_.

Step 8      JFPALOQSKR -> MISFORTUNE | SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
THIS IS AN O____ERMAN _RO_ER_.

Step 9      ZK -> AN        | SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
THIS IS AN O____ERMAN _RO_ER_.

Step 10     MOLTROE -> PROVERB | SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
THIS IS AN O____ERMAN PROVERB.

Step 11     LIUBROJZK -> OLDGERMAN | SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
THIS IS AN OLDGERMAN PROVERB.

```

Dictionary-assisted cracking time: 0.908 seconds

```

[10]:      cipher word #candidates      candidates      chosen
step
1          QDR          871  ABO, ABS, ABY, ACE, ACT, ADO, ...      THE
2          FP           59    AB, AD, AG, AI, AL, AM, ...      IS
3          QDFP          1          THIS      THIS
4      PRCSOFQX          3  SECURITY, SEDULITY, SENORITA  SECURITY
5          AFOPQ          1          FIRST      FIRST
6          CZSPR          1          CAUSE      CAUSE
7          LA           1          OF      OF
8      JFPALOQSKR          1  MISFORTUNE  MISFORTUNE
9          ZK           1          AN      AN
10         MOLTROE          1  PROVERB      PROVERB
11     LIUBROJZK          0          OLDGERMAN

```

```

[11]: SUBSTITUTION_KEY = dict(sorted(key.items())) # cipher letter -> plaintext letter
plain_to_cipher = {p: c for c, p in SUBSTITUTION_KEY.items()}

print("Plain :", " ".join(ALPHABET))
print("Cipher :", " ".join(plain_to_cipher.get(p, "?") for p in ALPHABET))

```

```

Plain : A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher : Z E C U R A B D F ? ? I J K L M ? O P Q S T ? ? X ?

```

The recovered cipher alphabet starts with Z E C U R and then continues in **alphabetical order** with the unused letters, so this is a *keyword substitution cipher* with keyword ZECUR(E) ("SECURE" with S replaced by Z). The keyword fills in the six letters that never appeared in the message.

```

[12]: KEYWORD = "ZECURE"
keyword_alphabet = "".join(dict.fromkeys(KEYWORD + ALPHABET)) # remove duplicated letters, keep_
order
assert all(keyword_alphabet[ALPHABET.index(p)] == c for p, c in plain_to_cipher.items())

FULL_KEY = dict(zip(keyword_alphabet, ALPHABET)) # cipher letter -> plaintext letter
PLAIN_TEXT = partial_decrypt(CIPHER_TEXT, FULL_KEY)

```

```
print("Plain :", " ".join(ALPHABET))
print("Cipher :", " ".join(keyword_alphabet))
print()
print("Cipher text:", CIPHER_TEXT)
print("Plain text :", PLAIN_TEXT)
```

Plain : A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
 Cipher : Z E C U R A B D F G H I J K L M N O P Q S T V W X Y

Cipher text: PRC5OFQX FP QDR AFOPQ CZSPR LA JFPALOQSKR. QDFP FP ZK LIUBROJZK
 MOLTROE.
 Plain text : SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLDGERMAN
 PROVERB.

1.2.6 Task (d): Brute-Force Attack on the Caesar Cipher

Design

- `caesar_encrypt(text, key)` shifts every letter key positions forward (non-letters are kept), and `caesar_decrypt` shifts backward.
- The Caesar key space has only 26 keys, so `brute_force_caesar()` simply decrypts the cipher text with **every** key.
- Each candidate is validated with the English dictionary: `dictionary_score` is the fraction of letters that belong to valid English words. Candidates are ranked by this score, and ties are broken by the chi-squared distance between the candidate's letter frequencies and English.
- The best candidate is accepted only if its dictionary score is at least `threshold` (0.8); otherwise the attack reports that the text is not a Caesar cipher.

```
[13]: KEY = 7 # secret shift of the Caesar cipher
MESSAGE = "SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB."

def caesar_encrypt(text, key):
    """Shift every letter 'key' positions forward in the alphabet; other characters are kept."""
    return "".join(ALPHABET[(ALPHABET.index(c) + key) % 26] if c in ALPHABET else c for c in text).upper()

def caesar_decrypt(text, key):
    return caesar_encrypt(text, -key)

caesar_cipher_text = caesar_encrypt(MESSAGE, KEY)
print(f"Key      : {KEY}")
print(f"Message    : {MESSAGE}")
print(f"Cipher text : {caesar_cipher_text}")
print(f"Decrypted   : {caesar_decrypt(caesar_cipher_text, KEY)}")
```

Key : 7
 Message : SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.
 Cipher text : ZLJBYPAF PZ AOL MPYZA JHBZL VM TPZMVYABUL. AOPZ PZ HU VSK NLYTHU WYVCLYI.
 Decrypted : SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.

```
[14]: def dictionary_score(text):
    """Fraction of letters that belong to valid English words (1.0 = every word is in the_
    dictionary)."""
    words = re.findall(r"[A-Z]+", text.upper())
    total = sum(len(w) for w in words)
```

```

    return sum(len(w) for w in words if w in ENGLISH_WORDS) / total if total else 0.0

def chi_squared(text):
    """Chi-squared distance from English letter frequencies (lower = more English-like)."""
    counts = Counter(c for c in text.upper() if c in ALPHABET)
    n = sum(counts.values())
    return sum((counts[c] - n * ENGLISH_FREQ[c] / 100) ** 2 / (n * ENGLISH_FREQ[c] / 100) for c
    in ALPHABET)

def brute_force_caesar(ciphertext, threshold=0.8):
    """Try all 26 keys; return the ranking and (key, plaintext) of the best candidate, or None."""
    rows = []
    for key in range(26):
        candidate = caesar_decrypt(ciphertext, key)
        rows.append((key, dictionary_score(candidate), chi_squared(candidate), candidate))

    ranking = pd.DataFrame(rows, columns=["key", "dictionary score", "chi-squared", "candidate_
    plaintext"])
    ranking = ranking.sort_values(["dictionary score", "chi-squared"], ascending=[False, True],
    ignore_index=True)

    best = ranking.iloc[0]
    answer = (int(best["key"]), best["candidate plaintext"]) if best["dictionary score"] >=
    threshold else None
    return ranking, answer

```

Demonstration 1: Crack the Caesar cipher text created above

```

[15]: ranking, answer = brute_force_caesar(caesar_cipher_text)
      recovered_key, recovered_text = answer

      print(f"Recovered key : {recovered_key}")
      print(f"Recovered text: {recovered_text}")
      assert recovered_key == KEY and recovered_text == MESSAGE

      ranking.round(3)

```

Recovered key : 7

Recovered text: SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN
PROVERB.

```

[15]:
  key  dictionary score  chi-squared \
0    7              1.000        15.347
1   14              0.102       573.317
2    1              0.068      1126.028
3   20              0.034       228.747
4   13              0.034       319.492
5    6              0.034       507.396
6    3              0.034       574.338
7    8              0.034       853.360
8   21              0.034       872.316
9   16              0.034      1539.437
10  19              0.000       193.239
11  23              0.000       241.563
12  11              0.000       346.692
13   4              0.000       385.701
14  24              0.000       423.847
15   5              0.000       484.029
16  17              0.000       544.654

```

17	18	0.000	562.324
18	22	0.000	643.940
19	15	0.000	750.566
20	10	0.000	773.703
21	9	0.000	941.817
22	12	0.000	1150.320
23	2	0.000	1232.451
24	0	0.000	1316.008
25	25	0.000	1567.501

candidate plaintext

```

0 SECURITY IS THE FIRST CAUSE OF MISFORTUNE. THIS IS AN OLD GERMAN PROVERB.
1 LXVNBKMR BL MAX YBKLM VTNLX HY FBLYHKMNGX. MABL BL TG HEW ZXKFTG IKHOXKU.
2 YKIAOXZE OY ZNK LOXYZ IGAYK UL SOYLUXZATK. ZNOY OY GT URJ MKXSGT VXUBKXH.
3 FRPHEVGL VF GUR SVEFG PNHFR BS ZVFSBEGHAR. GUVF VF NA BYQ TREZNA CEBIREO.
4 MYWOLCNS CM NBY ZCLMN WUOMY IZ GCMZILNOHY. NBCM CM UH IFX AYLGUH JLIPYLV.
5 TFDVSJUZ JT UIF GJSTU DBVTF PG NJTGPSUVOF. UIJT JT BO PME HFSNBO QSPWFSC.
6 WIGYVMXC MW XLI JMWX GEYWI SJ QMWJSVXYRI. XLMW MW ER SPH KIVQER TVSZIVF.
7 RDBTQHSX HR SGD EHQRS BZTRD NE LHRENSQSTMD. SGHR HR ZM NKC FDQLZM OQNUDQA.
8 EQOGDUFK UE FTQ RUDEF OMGEQ AR YUERADFGZQ. FTUE UE MZ AXP SQDYMZ BDAHQDN.
9 JVTILIZKP ZJ KYV WZIKJ TRLJV FW DZJWFIKLEV. KYZJ ZJ RE FCU XVIDRE GIFMVIS.
10 GSQIFWHM WG HVS TWFGH QOIGS CT AWGTCFHIBS. HVWG WG OB CZR USFAOB DFCJSFP.
11 COMEBSDI SC DRO PSBCD MKECO YP WSCPYBDEXO. DRSC SC KX YVN QOBWKX ZBYFOBL.
12 OAYQNEPU EO PDA BENOP YWQOA KB IEOBKNPQJA. PDEO EO WJ KHZ CANIWJ LNKRANX.
13 VHFUXULWB LV WKH ILUVW FDXVH RI PLVIRUWXQH. WKLW LV DQ ROG JHUPDQ SURYHUE.
14 BNLDARCH RB CQN ORABC LJDBN XO VRBOXACDWN. CQRB RB JW XUM PNAVJW YAXENAK.
15 UGEWTKVA KU VJG HKTUV ECWUG QH OKUHQTVWPG. VJKU KU CP QNF IGTQCP RTQXGTD.
16 IUSKHYJO YI JXU VYHIJ SQKIU EV CYIVEHJKDU. JXYI YI QD EBT WUHCQD FHELUHR.
17 HTRJGXIN XH IWT UXGHI RPJHT DU BXHUDGIJCT. IWXX XH PC DAS VTGBPC EGDKTGQ.
18 DPNFCTEJ TD ESP QTCDE NLFDP ZQ XTDQZCEFYF. ESTD TD LY ZWO RPCXLY ACZGPCM.
19 KWUMJALQ AK LZW XAJKL USMKW GX EAKXGJLMFW. LZAK AK SF GDV YWJESF HJGNWJT.
20 PBZROFQV FP QEB CFOPQ ZXRPB LC JFPCLOQRKB. QEFP FP XK LIA DBOJXK MOLSBOY.
21 QCASPGRW GQ RFC DGPQR AYSQC MD KGQDMPSRLC. RFGQ GQ YL MJB ECPKYL NPMTCPZ.
22 NZXPMDOT DN OCZ ADMNO XVPNZ JA HDNAJMOPIZ. OCDN DN VI JGY BZMHVI KMJQZMW.
23 XJHZWNVD NX YMJ KNWXY HFZXJ TK RNXTWYZSJ. YMNX NX FS TQI LJWRF S UWTJW.
24 ZLJBYPAF PZ AOL MPYZA JHBZL VM TPZMYABUL. AOPZ PZ HU VSK NLYTHU WYVCLYI.
25 AMKCZQBG QA BPM NQZAB KICAM WN UQANWZBCVM. BPQA QA IV WTL OMZUIV XZWMZJ.

```

[16]: # Timing: repeat the whole 26-key attack several times

```

N_RUNS = 100
runs = []
for _ in range(N_RUNS):
    t = time.perf_counter()
    brute_force_caesar(caesar_cipher_text)
    runs.append(time.perf_counter() - t)

print(f"Brute-force time over {N_RUNS} runs: "
      f"mean = {statistics.mean(runs) * 1000:.3f} ms, "
      f"stdev = {statistics.stdev(runs) * 1000:.3f} ms, "
      f"min = {min(runs) * 1000:.3f} ms, max = {max(runs) * 1000:.3f} ms")

```

Brute-force time over 100 runs: mean = 1.820 ms, stdev = 0.680 ms, min = 1.318 ms, max = 5.916 ms

[17]: # Every possible key is recovered correctly

```

for k in range(26):
    assert brute_force_caesar(caesar_encrypt(MESSAGE, k))[1][0] == k
print("All 26 keys recovered correctly.")

```

All 26 keys recovered correctly.

Demonstration 2: Run the same attack on the given cipher text

```
[18]: ranking, answer = brute_force_caesar(CIPHER_TEXT)
print(f"Result: {answer}")
ranking.head(5).round(3)
```

Result: None

```
[18]:      key  dictionary score  chi-squared  \
0    23           0.102    53.397
1    17           0.102   1149.848
2    25           0.085    907.408
3    22           0.068    509.034
4    10           0.034    376.513

                                candidate plaintext
0  SUFVRITA IS TGU DIRST FCVSU OD MISDORTVNU. TGIS IS CN OLXEURMCN PROWURH.
1  YALBXOZG OY ZMA JOXYZ LIBYA UJ SOYJUXZBTA. ZMOY OY IT URDKAXSIT VXUCAXN.
2  QSDTPGRY GQ RES BGPQR DATQS MB KGQBMPRTLS. REGQ GQ AL MJVCSPKAL NPMUSPF.
3  TVGWSJUB JT UHV EJSTU GDWTV PE NJTEPSUWOV. UHJT JT DO PMYFVSNDQ QSPXVSI.
4  FHSIEVGN VF GTH QVEFG SPIFH BQ ZVFQBEGIAH. GTVF VF PA BYKRHEZPA CEBJHEU.
```

```
[19]: per_key = statistics.mean(runs) / 26 # seconds to test one key
substitution_keys = math.factorial(26)

print(f"Caesar key space      : {26:>32,} keys -> {26 * per_key * 1000:.3f} ms")
print(f"Substitution key space : {substitution_keys:>32,} keys "
      f"-> {substitution_keys * per_key / (60 * 60 * 24 * 365):.2e} years")
```

Caesar key space : 26 keys -> 1.820 ms
 Substitution key space : 403,291,461,126,605,635,584,000,000 keys -> 8.95e+14 years

1.3 Problem 2: Cryptanalysis on Symmetric Encryption (Vigenère)

1.3.1 Vigenère Cipher

Each letter is shifted by the key letter at the same position, and the key is repeated over the whole message:
 $C_i = (P_i + K_{i \bmod m}) \bmod 26$, where m is the key length.

```
[20]: from collections import defaultdict

VIGENERE_KEY = "CRYPTO" # secret key of the Vigenere cipher
PLAIN_TEXT_2 = (
    "Cryptography has always been a contest between those who make codes and those who break them.
    "
    "For about three centuries the Vigenere cipher was known as the indecipherable cipher,
    because the same "
    "plaintext letter could be encrypted into many different ciphertext letters. A simple count
    of letters no "
    "longer revealed the secret, since every letter of the key used a different alphabet. In
    eighteen sixty three "
    "Friedrich Kasiski published a method that broke this cipher. He noticed that when a repeated
    word in the "
    "message lines up with the same part of the key, the ciphertext also repeats. The distance
    between the repeated "
    "groups is therefore a multiple of the length of the key. Once the length of the key is
    known, the ciphertext "
    "can be written in rows and the columns can be attacked one at a time, each column being
    nothing more than a "
    "Caesar cipher. The lesson for the modern security engineer is that a secret key must never
    be short and must "
```

```

    "never be reused in a predictable way, because every repetition gives the attacker a foothold.
    ↪ "
)

def only_letters(text):
    """Uppercase letters of `text`; spaces and punctuation are removed, as in classical cipher_
    ↪ texts."""
    return "".join(c for c in text.upper() if c in ALPHABET)

def vigenere_encrypt(text, key):
    """C_i = (P_i + K_(i mod m)) mod 26 over the letters of `text`."""
    letters = only_letters(text)
    return "".join(ALPHABET[(ALPHABET.index(c) + ALPHABET.index(key[i % len(key)])) % 26] for i,
    ↪ c in enumerate(letters))

def vigenere_decrypt(text, key):
    """P_i = (C_i - K_(i mod m)) mod 26."""
    return "".join(ALPHABET[(ALPHABET.index(c) - ALPHABET.index(key[i % len(key)])) % 26] for i,
    ↪ c in enumerate(text))

def groups(text, size=5, per_line=12):
    """Split text into blocks of `size` letters for printing."""
    blocks = [text[i:i + size] for i in range(0, len(text), size)]
    return "\n".join(" ".join(blocks[i:i + per_line]) for i in range(0, len(blocks), per_line))

VIGENERE_CIPHER_TEXT = vigenere_encrypt(PLAIN_TEXT_2, VIGENERE_KEY)
assert vigenere_decrypt(VIGENERE_CIPHER_TEXT, VIGENERE_KEY) == only_letters(PLAIN_TEXT_2)

print(f"Key      : {VIGENERE_KEY} (length {len(VIGENERE_KEY)})")
print(f"Letters : {len(VIGENERE_CIPHER_TEXT)}")
print("Cipher text:")
print(groups(VIGENERE_CIPHER_TEXT))

```

Key : CRYPTO (length 6)

Letters : 852

Cipher text:

```

EIWEM CIIYE AMJRQ PEKCP QQXSP RADGH GJRQX HYVCC MVQJC LACOR ITVCF VQPGR
VYMHX KJFZG XOMKF TFTQI YQHIV KFGXS EVLIN FKVQI ASXZE TGSTV AXIVG IUPLY
PFUCT GYXCX GRGTG EASTR ZAXQK GFTKP GTYJL SVYCH TAGGJ PBBVV VIESV KCGVC
WCBQX SPTPN IHGUG CMCOR LNWWH WCGXB VTGEA STKCM MZGKR TKGCJ GBIZG TMJGH
QWJTM HGIQC HZQEE TKFGM CPESF KFTLS EICIL WPTCT OSTPJ TMHGI MUMVG BCNNG
GUYSB THVPT GHCCN WTPGK GCXWI YRTXB UZVIR HJICT YFKVB GBQJB YHBGM ZNJUZ
KJFTW OOVRW HRVYV IUFOB CIAWU TGEAS TYCCH HKTCS MVCKU WXBCI CEXOV VBLHF
FZLIA SOVQH TUGCG CXGWG UXMVY YCHTA GGYGM CHKFT DSAKF TVWRY CGMSZ KYALC
TVNTT HUKFT WWUKY CVSDV RLXSP KFTKS RVYIX RIIMJ IGKJR WXFOW MGXOO LJIBD
NVMUM VGCCC ZHJFD IASMV WDGQG KFTES PXRWH TVYCZ XMKJI CHPKP FTVWR YCGMS
ZKAPG PGNPX MHGEG CKCYJ YCWHJ VADEI OEQRT BDVYI MOEBC SHBGR RPMWO VCPVY
EFJJF BDVGC ZBQKF XGUOF PTMVC EYRTS URPRB DJVPI ASNVQ HHBHF PIASO FBTKB
UVAJK WVPCC ZWPVC GBGVY YITGG TPTMY GPKJL HPVTT KPGJF DKHCE BBNVG ECKXF
DVPTN GGUGC TDTVB XVHCS JTPOA SCRTI UVCKX FAICE XHKKG DGUKM CHMVG RRITQ
MVPPY CQKFD ER

```

[21]: # Small example: the same plaintext letter becomes different cipher letters

```

example = only_letters("ATTACK AT DAWN")
print("Plain :", " ".join(example))
print("Key   :", " ".join((VIGENERE_KEY * len(example))[:len(example)]))

```

```
print("Cipher :", " ".join(vigenere_encrypt(example, VIGENERE_KEY)))
```

```
Plain  : A T T A C K A T D A W N
Key    : C R Y P T O C R Y P T O
Cipher : C K R P V Y C K B P P B
```

1.3.2 Letter Statistics: Why Plain Frequency Analysis Fails

The *index of coincidence* (IC) is the probability that two letters picked at random from a text are equal. English text has IC = 0.066, while uniformly random letters have IC = $1/26 \approx 0.038$.

```
[22]: def index_of_coincidence(text):
    """Probability that two letters picked at random from `text` are the same."""
    counts = Counter(text)
    n = len(text)
    return sum(f * (f - 1) for f in counts.values()) / (n * (n - 1))

def top_letters(text, k=5):
    counts = Counter(text)
    return " ".join(f"{c} {100 * n / len(text):.1f}%" for c, n in counts.most_common(k))

IC_ENGLISH = sum((p / 100) ** 2 for p in ENGLISH_FREQ.values())
IC_RANDOM = 1 / 26

plain_letters = only_letters(PLAIN_TEXT_2)
stats_df = pd.DataFrame(
    {
        "index of coincidence": [IC_ENGLISH, IC_RANDOM, index_of_coincidence(plain_letters),
        ↪ index_of_coincidence(VIGENERE_CIPHER_TEXT)],
        "top 5 letters": ["-", "-", top_letters(plain_letters),
        ↪ top_letters(VIGENERE_CIPHER_TEXT)],
    },
    index=["English (expected)", "random (expected)", "plain text", "Vigenere cipher text"],
)
stats_df.round(4)
```

```
[22]: index of coincidence \
English (expected)          0.0655
random (expected)          0.0385
plain text                  0.0784
Vigenere cipher text       0.0463

                                top 5 letters
English (expected)            -
random (expected)             -
plain text                    E 18.7%, T 11.2%, A 6.7%, H 6.7%, R 6.5%
Vigenere cipher text          G 8.5%, C 7.7%, V 7.2%, T 6.9%, K 5.2%
```

1.3.3 Step 1: Kasiski Examination (Find the Key Length)

If a piece of plaintext repeats **and** both copies start at the same position of the key, the cipher text repeats too. The distance between the two copies is then a multiple of the key length, so the most common factors of the distances point to the key length.

```
[23]: def repeated_sequences(ciphertext, min_len=3, max_len=20):
    """Maximal repeated substrings (at least `min_len` letters) and the positions where they
    ↪ occur."""
    seen = defaultdict(list)
```

```

for length in range(min_len, max_len + 1):
    for i in range(len(ciphertext) - length + 1):
        seen[ciphertext[i:i + length]].append(i)
repeats = {s: pos for s, pos in seen.items() if len(pos) > 1}

def extends(s, pos):
    # s is only part of a longer repeat that occurs at exactly the same places
    return any(len(repeats.get(s + c, ())) == len(pos) or len(repeats.get(c + s, ())) == len(pos) for c in ALPHABET)

    return {s: pos for s, pos in repeats.items() if not extends(s, pos)}

def factors(n, limit=20):
    return [f for f in range(2, limit + 1) if n % f == 0]

repeats = repeated_sequences(VIGENERE_CIPHER_TEXT)
kasiski_df = pd.DataFrame(
    [
        (seq, len(seq), pos, [b - a for a, b in zip(pos, pos[1:])])
        for seq, pos in sorted(repeats.items(), key=lambda kv: (-len(kv[0]), kv[1][0]))
    ],
    columns=["sequence", "length", "positions", "distances"],
)
kasiski_df["factors (2-20)"] = kasiski_df["distances"].map(lambda ds: sorted({f for d in ds for f_ in factors(d)}))
kasiski_df.index += 1

long_repeats = kasiski_df[kasiski_df["length"] >= 4]
print(f"Repeated sequences: {len(kasiski_df)} in total, {len(long_repeats)} with at least 4 letters")
long_repeats

```

Repeated sequences: 57 in total, 15 with at least 4 letters

```

[23]:
sequence length positions distances \
1 KFTVWRYCGMSZK 13 [463, 589] [126]
2 VYCHTAGG 8 [156, 444] [288]
3 TGEAST 6 [133, 211, 385] [78, 174]
4 JTMHGI 6 [242, 284] [42]
5 MUMVG 5 [290, 542] [252]
6 QXSP 4 [21, 183] [162]
7 TKPG 4 [147, 759] [612]
8 GUGC 4 [192, 786] [594]
9 NGGU 4 [298, 784] [486]
10 VYYI 4 [372, 738] [366]
11 ICEX 4 [409, 817] [408]
12 IASO 4 [423, 711] [288]
13 PKFT 4 [504, 588] [84]
14 PIAS 4 [698, 710] [12]
15 CKXF 4 [776, 812] [36]

```

```

factors (2-20)
1 [2, 3, 6, 7, 9, 14, 18]
2 [2, 3, 4, 6, 8, 9, 12, 16, 18]
3 [2, 3, 6, 13]
4 [2, 3, 6, 7, 14]
5 [2, 3, 4, 6, 7, 9, 12, 14, 18]
6 [2, 3, 6, 9, 18]
7 [2, 3, 4, 6, 9, 12, 17, 18]

```

```

8          [2, 3, 6, 9, 11, 18]
9          [2, 3, 6, 9, 18]
10         [2, 3, 6]
11        [2, 3, 4, 6, 8, 12, 17]
12 [2, 3, 4, 6, 8, 9, 12, 16, 18]
13        [2, 3, 4, 6, 7, 12, 14]
14         [2, 3, 4, 6, 12]
15        [2, 3, 4, 6, 9, 12, 18]

```

```

[24]: def factor_counts(distances, limit=20):
        counts = Counter(f for d in distances for f in factors(d, limit))
        return pd.Series({f: counts[f] for f in range(2, limit + 1)})

all_distances = [d for ds in kasiski_df["distances"] for d in ds]
long_distances = [d for ds in long_repeats["distances"] for d in ds]

factor_df = pd.DataFrame(
    {
        "all repeats (>= 3 letters)": factor_counts(all_distances),
        "repeats with >= 4 letters": factor_counts(long_distances),
    }
)
factor_df.index.name = "factor"

short_repeats = kasiski_df[kasiski_df["length"] == 3]
accidental = short_repeats["distances"].map(lambda ds: any(d % len(VIGENERE_KEY) for d in ds)).
    sum()

print(f"GCD of all distances          : {math.gcd(*all_distances)}")
print(f"GCD of distances (>= 4 letters) : {math.gcd(*long_distances)}")
print(f"3-letter repeats that are accidental (checked with the real key): {accidental} of_
    {len(short_repeats)}")
factor_df.T

```

```

GCD of all distances          : 1
GCD of distances (>= 4 letters) : 6
3-letter repeats that are accidental (checked with the real key): 19 of 42

```

```

[24]: factor          2  3  4  5  6  7  8  9  10 11 12 13 \
all repeats (>= 3 letters) 61 58 30 12 54 11 12 23  8  5 28  5
repeats with >= 4 letters  16 16  8  0 16  4  3  9  0  1  8  1

factor          14 15 16 17 18 19 20
all repeats (>= 3 letters) 11  8  7  5 22  5  3
repeats with >= 4 letters  4  0  2  2  9  0  0

```

1.3.4 Step 2: Superimposition

- **Superimposition test (confirm the key length):** place the cipher text over a copy of itself shifted by s letters and count the positions where the two letters are equal. When s is a multiple of the key length, both letters were encrypted with the same key letter, so the rate is close to English (6.6%); otherwise it is close to random (3.8%).
- **Superimposition by columns (Kasiski's method):** write the cipher text in rows of m letters, i.e. stack copies shifted by m . Every column was then encrypted with a single key letter, so each column is an ordinary Caesar cipher.

```

[25]: def coincidence_rate(ciphertext, shift):
        """Superimpose the cipher text on itself shifted by `shift`; percentage of aligned letters_
        that are equal."""

```

```

pairs = list(zip(ciphertext, ciphertext[shift:]))
return 100 * sum(a == b for a, b in pairs) / len(pairs)

coincidence_df = pd.DataFrame(
    [(s, coincidence_rate(VIGENERE_CIPHER_TEXT, s)) for s in range(1, 25)],
    columns=["shift", "coincidence rate (%)"],
).set_index("shift")

print(f"Expected rate: English {100 * IC_ENGLISH:.2f}%, random {100 * IC_RANDOM:.2f}%")
display(coincidence_df.round(2).T.iloc[:, :12])
display(coincidence_df.round(2).T.iloc[:, 12:])

```

Expected rate: English 6.55%, random 3.85%

shift	1	2	3	4	5	6	7	8	9	10 \
coincidence rate (%)	4.7	4.47	4.0	3.3	5.19	6.74	3.55	4.62	5.1	3.56

shift	11	12
coincidence rate (%)	3.33	8.33

shift	13	14	15	16	17	18	19	20	21 \
coincidence rate (%)	4.41	3.58	5.14	4.31	4.19	8.03	3.84	3.61	2.65

shift	22	23	24
coincidence rate (%)	3.98	4.83	7.73

```

[26]: def average_column_ic(ciphertext, length):
    """Mean IC of the columns obtained by writing the cipher text in rows of `length` letters."""
    return statistics.mean(index_of_coincidence(ciphertext[i::length]) for i in range(length))

column_ic_df = pd.DataFrame(
    [(m, average_column_ic(VIGENERE_CIPHER_TEXT, m)) for m in range(1, 13)],
    columns=["key length", "average column IC"],
).set_index("key length")
column_ic_df.round(4).T

```

```

[26]: key length      1      2      3      4      5      6      7 \
average column IC  0.0463  0.0508  0.0596  0.0511  0.0463  0.0776  0.0462

key length      8      9      10      11      12
average column IC  0.0507  0.0598  0.0513  0.0444  0.0801

```

```

[27]: def guess_key_length(ciphertext, max_len=12, ratio=0.9):
    """Smallest length whose columns look (almost) as English as the best length; multiples of_
    ↳ the key score alike."""
    scores = {m: average_column_ic(ciphertext, m) for m in range(1, max_len + 1)}
    best = max(scores.values())
    return min(m for m, score in scores.items() if score >= ratio * best)

peak_shifts = [s for s in coincidence_df.index if coincidence_df.loc[s, "coincidence rate (%)"]_
    ↳ >= 0.9 * 100 * IC_ENGLISH]
key_length = guess_key_length(VIGENERE_CIPHER_TEXT)
print(f"Shifts with an English-like coincidence rate: {peak_shifts} (GCD = {math._
    ↳ gcd(*peak_shifts)})")
print(f"Smallest length with an English-like average column IC: {key_length}")

```

Shifts with an English-like coincidence rate: [6, 12, 18, 24] (GCD = 6)

Smallest length with an English-like average column IC: 6

```
[28]: rows = [VIGENERE_CIPHER_TEXT[i:i + key_length] for i in range(0, len(VIGENERE_CIPHER_TEXT),
        ↪key_length)]
print(f"Cipher text written in rows of {key_length} letters (first 10 of {len(rows)} rows):")
print("column " + " ".join(str(i + 1) for i in range(key_length)))
for r in rows[:10]:
    print("      " + " ".join(r))
```

Cipher text written in rows of 6 letters (first 10 of 142 rows):

```
column  1 2 3 4 5 6
        E I W E M C
        I I Y E A M
        J R Q P E K
        C P Q Q X S
        P R A D G H
        G J R Q X H
        Y V C C M V
        Q J C L A C
        O R I T V C
        F V Q P G R
```

1.3.5 Step 3: Solve Every Column as a Caesar Cipher

Each column is decrypted with all 26 shifts, and the shift whose letter frequencies are closest to English (lowest chi-squared, from Problem 1) gives one key letter.

```
[29]: def solve_column(column):
        """(chi-squared, shift) of all 26 Caesar shifts of one column, best first."""
        return sorted((chi_squared(caesar_decrypt(column, s)), s) for s in range(26))

column_rows = []
for i in range(key_length):
    column = VIGENERE_CIPHER_TEXT[i::key_length]
    (best_chi, best_shift), (second_chi, _) = solve_column(column)[:2]
    most_common = Counter(column).most_common(1)[0][0]
    column_rows.append((
        i + 1, len(column), most_common, ALPHABET[(ALPHABET.index(most_common) - ALPHABET.
        ↪index("E")) % 26],
        ALPHABET[best_shift], best_chi, second_chi,
    ))

column_df = pd.DataFrame(
    column_rows,
    columns=["column", "letters", "most common", "key if most common = E", "key (chi-squared)",
    ↪"chi-squared (best)", "chi-squared (2nd)"],
).set_index("column")

recovered_vigenere_key = "".join(column_df["key (chi-squared)"])
print(f"Recovered key: {recovered_vigenere_key}")
column_df.round(2)
```

Recovered key: CRYPTO

```
[29]:          letters most common key if most common = E key (chi-squared) \
column
1          142          G          C          C
2          142          V          R          R
3          142          C          Y          Y
4          142          T          P          P
5          142          X          T          T
```

6	142	S	0	0
	chi-squared (best)	chi-squared (2nd)		
column				
1	48.99	369.10		
2	46.09	380.07		
3	44.34	462.03		
4	37.81	314.22		
5	14.49	386.01		
6	14.35	481.29		

```
[30]: recovered_plain_2 = vigenere_decrypt(VIGENERE_CIPHER_TEXT, recovered_vigenere_key)
      assert recovered_vigenere_key == VIGENERE_KEY
      assert recovered_plain_2 == only_letters(PLAIN_TEXT_2)
      print(groups(recovered_plain_2))
```

CRYPT OGRAP HYHAS ALWAY SBEEN ACONT ESTBE TWEEN THOSE WHOMA KECOD ESAND
 THOSE WHOBR EAKTH EMFOR ABOUT THREE CENTU Riest HEVIG ENERE CIPHE RWASK
 NOWNA STHEI NDECI PHERA BLECI PHERB ECAUS ETHES AMEPL AINTE XTLET TERCO
 ULDBE ENCRY PTEDI NTOMA NYDIF FEREN TCIPH ERTEX TLETT ERSAS IMPL E COUNT
 OFLET TERN OLONG ERREV EALD THESE CRETS INCDE VERYL ETTER OFTHE KEYUS
 EDADI FFERE NTALP HABET INEIG HTEEN SIXTY THREE FRIED RICHK ASISK IPUBL
 ISHD AMETH ODTA TBROK ETHIS CIPHE RHENO TICED THATW HENAR EPEAT EDWOR
 DINTH EMESS AGELI NESUP WITHT HESAM EPART OFTHE KEYTH ECIPH ERTEX TALSO
 REPEA TSTHE DISTA NCEBE TWEEN THERE PEATE DGROU PSIST HEREF OREAM ULTIP
 LEOFT HELEN GTHOF THEKE YONCE THELE NGTHO FTHEK EYISK NOWNT HECIP HERTE
 XTCAN BEWRI TTENI NROWS ANDTH ECOLU MNSCA NBEAT TACK E DONEA TATIM EEACH
 COLUM NBEIN GNOTH INGMO RETHA NACAE SARCI PHERT HELES SONFO RTHEM ODERN
 SECUR ITYEN GINEE RISTH ATASE CRETK EYMUS TNEVE RBESH ORTAN DMUST NEVER
 BEREU SEDIN APRED ICTAB LEWAY BECAU SEEVE RYREP ETITI ONGIV ESTHE ATTAC
 KERA F OOTH O LD

1.3.6 Complete Attack and Its Limits

```
[31]: def kasiski_attack(ciphertext):
      """Find the key length by superimposition, then solve every column as a Caesar cipher."""
      length = guess_key_length(ciphertext)
      key = "".join(ALPHABET[solve_column(ciphertext[i::length])[0][1]] for i in range(length))
      return key, vigenere_decrypt(ciphertext, key)

      N_RUNS = 20
      attack_runs = []
      for _ in range(N_RUNS):
          t = time.perf_counter()
          key, _ = kasiski_attack(VIGENERE_CIPHER_TEXT)
          attack_runs.append(time.perf_counter() - t)
      assert key == VIGENERE_KEY

      print(f"Recovered key: {key}")
      print(f"Attack time over {N_RUNS} runs: mean = {statistics.mean(attack_runs) * 1000:.2f} ms, "
            f"stdev = {statistics.stdev(attack_runs) * 1000:.2f} ms")
```

Recovered key: CRYPTO

Attack time over 20 runs: mean = 9.15 ms, stdev = 1.11 ms

```
[32]: # How much cipher text does the attack need?
      limit_rows = []
      for n in [60, 120, 240, 480, len(VIGENERE_CIPHER_TEXT)]:
          prefix = VIGENERE_CIPHER_TEXT[:n]
```

```
key, _ = kasiski_attack(prefix)
correct_letters = sum(a == b for a, b in zip(key, VIGENERE_KEY)) if len(key) == _
len(VIGENERE_KEY) else 0
limit_rows.append((n, n // len(key), len(key), key, f"{correct_letters}/{len(VIGENERE_KEY)}"))

pd.DataFrame(limit_rows, columns=["letters", "letters per column", "key length found", "recovered_
key", "correct key letters"]).set_index("letters")
```

[32]:

	letters per column	key length found	recovered key	\
letters				
60	10	6	CRCLTO	
120	20	6	CRYPTO	
240	40	6	CRYPTO	
480	80	6	CRYPTO	
852	142	6	CRYPTO	

	correct key letters
letters	
60	4/6
120	6/6
240	6/6
480	6/6
852	6/6